

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Queued	3
Feature	Queued	2
Task	Queued	4
TOTAL		9

Items

ATM ID	Type	Status	Severity	Description
HXC-118	Feature	Queued	High	A dedicated Retrieval-Augmented-Generation component is maintained as its own reusable module, but the main application does not import or use it anywhere. A capability the product is expected to offer (answering using retrieved documents) is therefore effectively unavailable to end users despite the code existing. The work integrates the existing RAG module into the application, wires it into the request flow, and exposes its capability flag. Users gain working document-grounded answers instead of an orphaned, unused component.

ATM ID	Type	Status	Severity	Description
HXC-119	Feature	Queued	High	Governance rule CONST-040 lists the Agent Client Protocol among required capabilities, but there is no implementation of it anywhere in the codebase. Any user or integration expecting ACP connectivity currently cannot use it. The work is to design and implement real ACP support, or, if it proves structurally infeasible, to document that with cited evidence. The platform will then either genuinely support ACP or hold an honest, evidenced position instead of an unmet claim.
HXC-122	Task	Queued	Medium	Two categories of tests, memory-usage and end-to-end automation, skip most of their cases by default because they require a live server or special environment flags not set in normal runs. In practice these areas are largely unverified even though the tests appear to exist. The work provides a documented, repeatable way to run them against real infrastructure so they actually execute and prove the behavior. Memory and automation behavior then becomes genuinely tested rather than merely scaffolded.
HXC-136	Task	Queued	Medium	Several mandated automated test categories — load/denial-of-service, scaling, stress and chaos, and user-interface/experience — were not exercised in the latest real-infrastructure run, so their current health is unconfirmed. The work is to run each of these test types against real infrastructure and capture proof of the results. This completes the promised full

ATM ID	Type	Status	Severity	Description
				test-type coverage and confirms the product holds up under load and adverse conditions.
HXC-138	Task	Queued	Low	The end-to-end challenge runner can now launch all its scenarios (a missing option was just fixed), but the scenarios still need to be executed against a live server with a real model to confirm the complete user journeys work. The work is to stand up a server and run the challenges, capturing the results. This provides real proof that the headline user workflows function end to end.
HXC-144	Bug	Queued	Medium	Under the sustained request-flood chaos test against the real running server, the goroutine count grew by 5 which exceeds the tolerance of 4, signalling a goroutine leak in a request-handling path when the server is hammered. Left unaddressed this degrades long-running server stability under load. The work is to find the leaking goroutine (likely an unclosed channel, context, or connection in a hot handler) and fix it so the count stays within tolerance under flood. Evidence: docs/qa/infra_retest_20260712_hxc122_138/EVIDENCE.md (Server 7/8).
HXC-145	Bug	Queued	Low	During the real-infra retest the Xiaomi provider chaos tests failed 2 of 5 because the model id configured for Xiaomi (mimo-v2-flash) is rejected by the live Xiaomi API, indicating the configured model name is stale or wrong. Users selecting the Xiaomi provider with that model would

ATM ID	Type	Status	Severity	Description
				get errors. The work is to determine the correct current Xiaomi model id (from the provider or the verifier as single source of truth) and update the configuration so Xiaomi requests succeed. Evidence: docs/qa/infra_retest_20260712_hxc122_138/EVIDENCE.md (Xiaomi 3/5). The e2e challenge runner advertises multiple interface modes (cli, rest, tui, websocket) but none of them actually exercises the HelixCode server's real HTTP API during a run, so the challenges validate the runner's own logic rather than the shipped server endpoints. This is a documentation-versus-implementation gap that weakens the end-to-end proof. The work is to wire the challenge runner's interfaces to genuinely call the running server's HTTP API so the challenges prove the real user-facing endpoints work. Discovered 2026-07-12 real-infra retest. Evidence: docs/qa/infra_retest_20260712_hxc122_138/hxc138_challenge_report.json. Running the (now-compilable) automation test binary against the live OpenRouter API, TestAllFreeProvidersAutomation Provider_OpenRouter BasicGeneration panics with a nil-pointer dereference: the configured free model id deepseek-r1-free is stale/rejected and the code path is missing a nil-check on the error before using the response. Users of the
HXC-146	Task	Queued	Medium	
HXC-147	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				OpenRouter free provider with that model would hit the same crash. The work is to correct the free-provider model id (sourced from the verifier as single source of truth) and add the missing nil-check so a rejected model degrades gracefully instead of panicking. NOTE this environment has live provider API keys set so provider tests spend real money; guard/skip accordingly. Found 2026-07-12.